

JSIntel

JavaScript Intelligence, Analysis, Correlation, and Knowledge Platform

Architecture, Implementation, Research Direction, and Future Enhancements

REPOSITORY	github.com/maxamin/jsintel
AUTHOR / MAINTAINER	maxamin
PROJECT STATUS	Phase 1 skeleton shipped; Phase 2 plugin/AI evolution in design
AUDIENCE	Security engineers, architects, developers, researchers, AppSec teams
SCOPE	Authorized security testing, defensive asset inventory, software assurance
DOCUMENT VERSION	2.0

1. Executive Summary

JSIntel began as a JavaScript crawling and extraction utility: point it at a scope, pull down .js assets, and grep them for URLs and secrets. That framing undersells what the architecture has grown into. JSIntel is evolving into an application intelligence platform that reconstructs the behavior of a client-side application from its shipped artifacts — not just what strings appear in a file, but how modules import one another, how functions reach network calls, which endpoints those calls resolve to, how that surface changes between scans, and, where AST and regex evidence run out, what a JavaScript-specific language model can infer about intent from the full, deobfuscated asset tree.

The distinction matters operationally. A crawler produces a list of files. An intelligence platform produces a queryable model of an application: assets contain modules, modules define functions, functions reach endpoints, and every one of those relationships carries provenance and a confidence score. That model is what makes correlation, historical diffing, automatic API documentation, and CI/CD gating possible — none of which are reachable from a flat list of regex hits.

This document describes JSIntel's Phase 1 skeleton as shipped, the Phase 2 architecture (plugin system, AST analysis, knowledge graph, runtime intelligence, AI-assisted semantic analysis) as designed, and the research and product direction beyond Phase 2: taint tracking, source map recovery, automatic OpenAPI and Insomnia collection generation, a REST/GraphQL API layer, and CI/CD integration. Diagrams throughout distinguish shipped components (solid borders) from planned components (dashed borders).

1.1 Phase 1 Skeleton (Shipped)

- Katana-based, JavaScript-aware crawling with HTTPX/GAU/Wayback fallback adapters.
- Asset classification, URL normalization, and parallel resumable downloads.
- SHA-256 / ETag integrity metadata captured per asset.
- Regex-based extraction of URLs, endpoints, imports, WebSocket references, and framework signatures.
- SQLite-backed storage with JSON, Markdown, and CSV reporting.

1.2 Phase 2 Evolution (Designed)

- A plugin architecture with a stable four-method contract for all analyzers.
- Tree-sitter based AST analysis replacing regex as the primary extraction mechanism.
- A relational knowledge graph linking assets, modules, functions, endpoints, and dependencies.
- Runtime intelligence via Playwright-driven browser instrumentation.
- An AI-assisted semantic analysis stage operating on deobfuscated source.

2. System Architecture

Figure 1 shows the full component architecture across ten cooperating layers: the CLI entry point, the collection layer, the asset management pipeline, the planned AI deobfuscation and semantic layer, the plugin-based analysis engine, runtime intelligence, the SQLite/PostgreSQL knowledge layer, the correlation and risk-scoring intelligence layer, the reporting layer, and supporting external systems. Dashed borders mark components that are designed but not yet part of the Phase 1 skeleton.

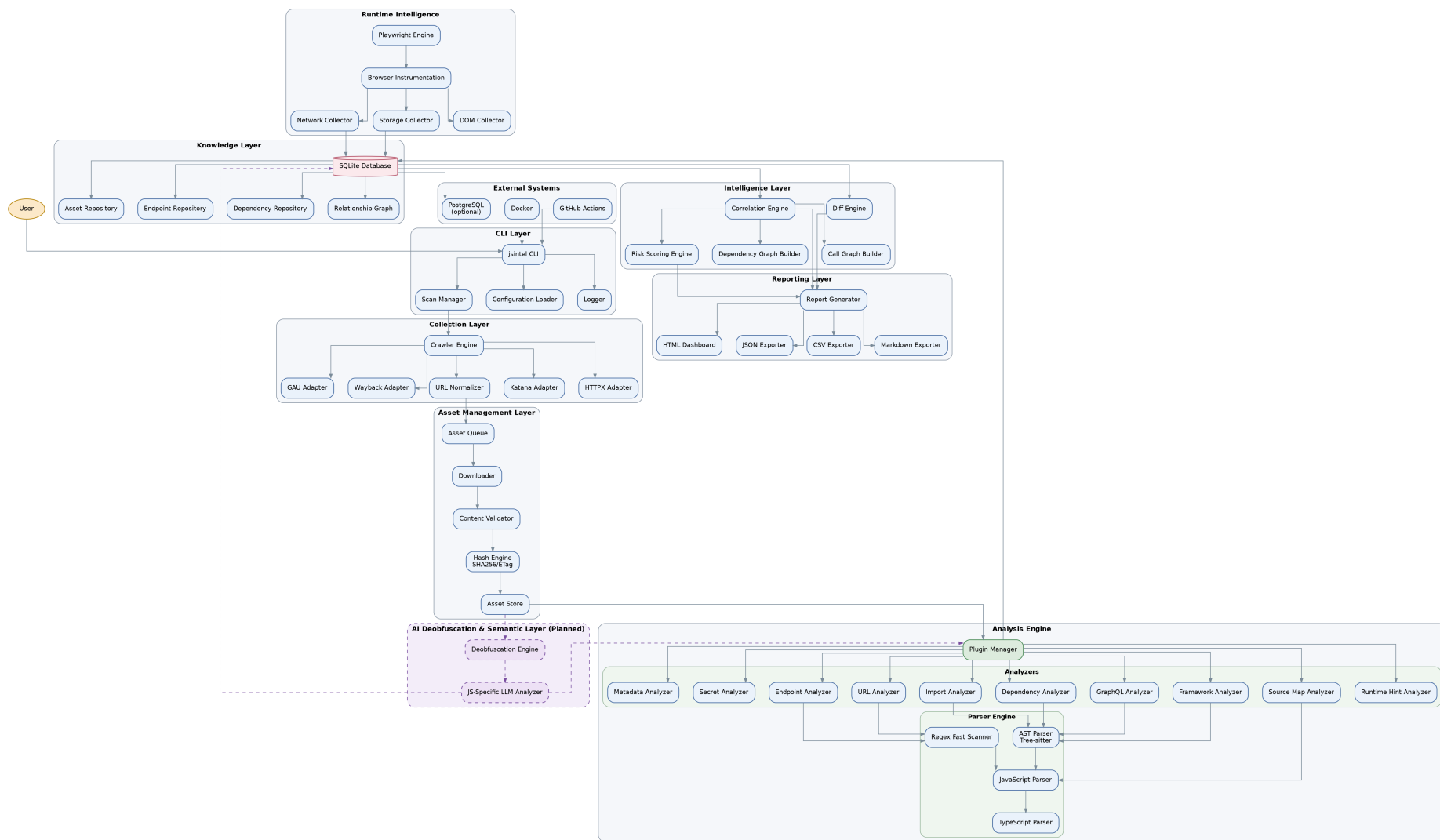


Figure 1. JSIntel end-to-end component architecture (UML component diagram).

3. Advanced Analysis Engine

The analysis engine's job is to turn a stored asset into typed, queryable facts. Phase 1 does this with regular expressions against raw source text, which is fast but shallow: it cannot distinguish a URL inside a string literal from one inside a comment, and it cannot follow a value across variable assignments. Phase 2 replaces this with structural analysis over a real parse tree.

3.1 AST Intelligence

Parsing is performed with Tree-sitter grammars for JavaScript, TypeScript, JSX, and TSX, giving every analyzer plugin a concrete syntax tree instead of a string buffer. Tree-sitter's incremental parsing model also means re-parsing a changed file on a subsequent scan is proportional to the size of the edit, not the size of the file — which matters at repository scale.

From each parse tree, the AST analyzers extract:

- Imports and exports (ES modules, CommonJS `require()`, dynamic `import()`).
- Function and class declarations, including arrow functions and class methods.
- Dynamic imports and code-splitting boundaries (chunk entry points).
- Fetch/XHR/axios call sites and their argument expressions.
- WebSocket constructor calls and message handler registration.
- String-built API paths (template literals and concatenation feeding a network call).

3.2 Cross-Asset Symbol Resolution

A single bundled file is rarely the unit of analysis a security reviewer cares about; the question is usually “what does this application do,” which spans many files. JSIntel resolves symbols across asset boundaries by building four linked graphs: a module graph (which files import which), a dependency graph (which third-party packages are pulled in), a chunk graph (which bundle chunks are lazy-loaded from where), and a call graph (which functions invoke which). Chaining these turns a pile of individually-parsed files into one application model, illustrated in Figure 2.



Figure 2. Cross-asset resolution chain: Asset → Module → Function → Network Call → Endpoint.

3.3 Taint Tracking Engine

The taint engine performs source-to-sink analysis over the AST: it marks values originating from attacker-influenceable sources, propagates that taint through transformations, and reports where a tainted value reaches a dangerous sink. This is the mechanism behind DOM XSS and client-side injection findings that pattern matching alone cannot support, because it requires reasoning about data flow rather than surface syntax.

Sources tracked include:

- URL parameters (`location.search`, `URLSearchParams`)
- Location APIs (`location.hash`, `location.href`)
- `postMessage` event data
- Cookies (`document.cookie`)

- Web Storage (localStorage, sessionStorage)

Sinks tracked include:

- innerHTML / outerHTML assignment
- eval()
- The Function() constructor
- Dangerous DOM operations (document.write, insertAdjacentHTML)

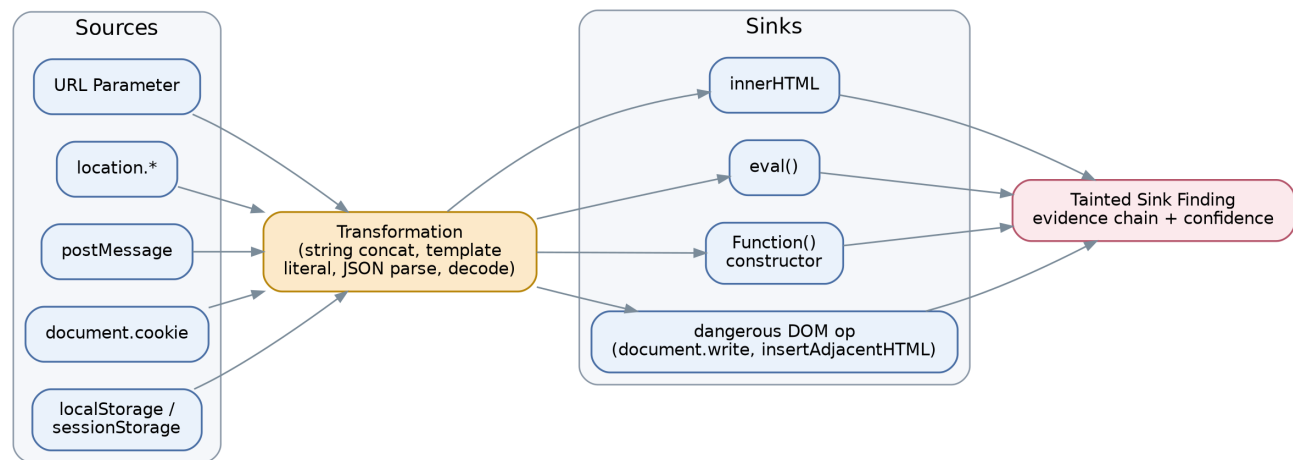


Figure 3. Source → transformation → sink taint propagation model.

Each finding carries an evidence chain — the concrete AST path from source to sink, including every intermediate transformation — and a confidence score reflecting how directly the source reaches the sink versus how much the value is reshaped along the way. See Section 7 for how this score composes with other evidence types.

3.4 Source Map Intelligence

When a target exposes source maps alongside its bundles, JSIntel reconstructs the original, pre-build source tree rather than analyzing only the minified output. This recovers original filenames, function and class names, inline comments, original TypeScript source (when the map references .ts/.tsx sources), and often enough directory structure to infer the target's internal architecture.

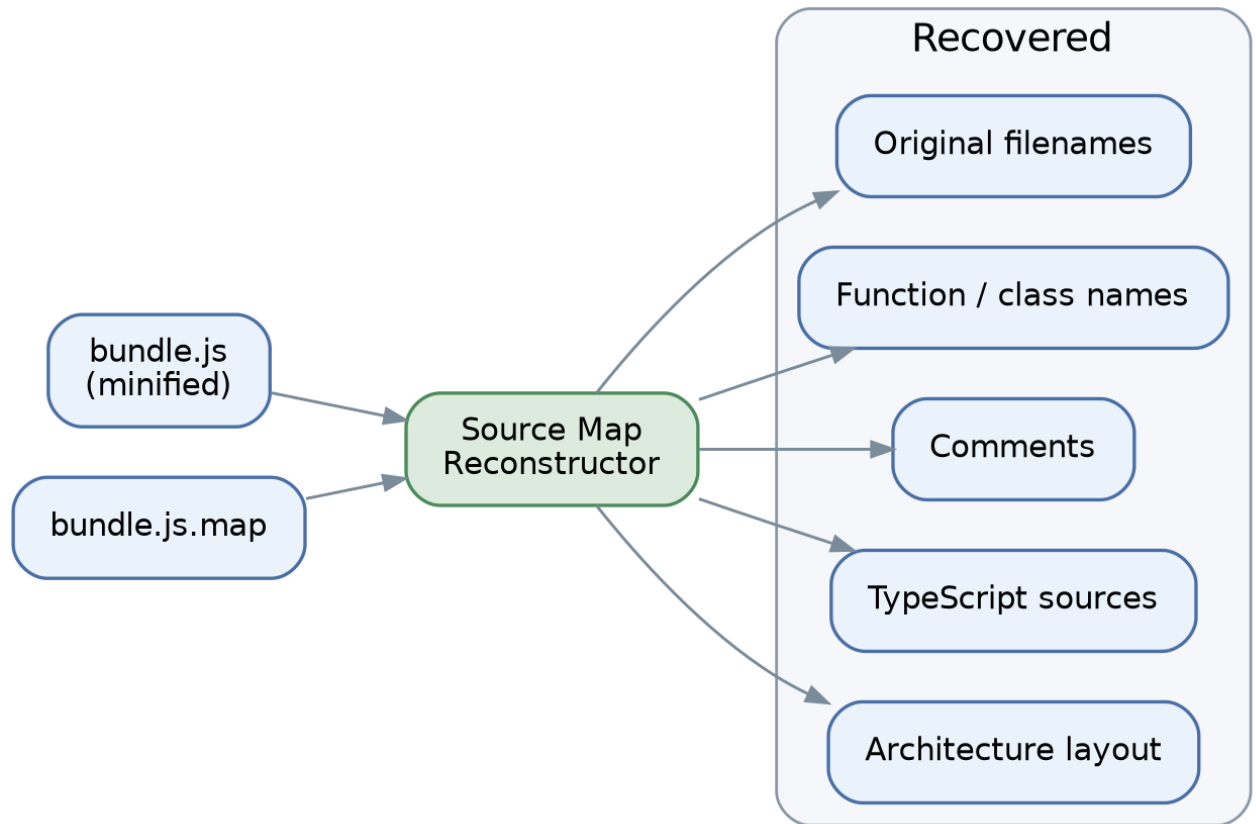


Figure 4. Source map recovery: `bundle.js + bundle.js.map` → original source tree.

4. AI Intelligence Layer

The AI layer (Section 2, dashed) sits after deobfuscation and before — not instead of — the plugin-based analyzers. Two design constraints shape it: cost has to scale sub-linearly with repeated scans of a mostly-unchanged target, and its output has to be consumable by the same pipeline that consumes regex and AST findings.

4.1 Diff-Aware AI Analysis

Running a full-tree LLM pass on every scan is not economical for continuous monitoring. Instead, the analyzer computes a targeted change set from the previous scan's snapshot, the AST nodes that changed, and the assets that are new or modified, and analyzes only that set:

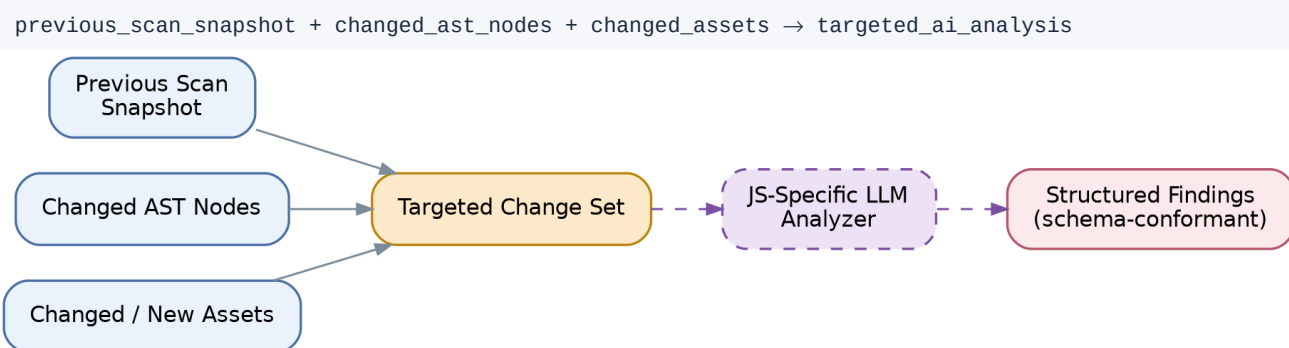


Figure 5. Diff-aware AI analysis: only the changed surface is re-analyzed by the LLM.

This targeting gives three practical benefits:

- Reduced cost — token spend scales with the size of a diff, not the size of the codebase.
- Faster continuous analysis — incremental scans complete in a small fraction of a full-tree pass.
- Scalable monitoring — makes scheduled or on-commit AI analysis economically viable at scale.

4.2 Structured AI Findings

The LLM analyzer's output must conform to the same plugin finding schema used by every other analyzer, so results merge into risk scoring and reporting without a special case. A finding is a typed record with a source tag identifying how it was derived:

```
{ "type": "endpoint", "confidence": 0.96, "source": "AST+AI", "value": "/api/users" }
```

This structure lets the Plugin Manager, Risk Scoring Engine, and Report Generator treat an AI-derived finding exactly like a regex- or AST-derived one — the only difference is the provenance recorded in source and the confidence it contributes (Section 7).

5. Knowledge Graph

The knowledge layer is modeled as a graph, not a flat table dump. Every entity discovered during a scan — assets, modules, functions, endpoints, dependencies, runtime requests, and findings — is a node, and the relationships between them (contains, defines, reaches, imports, observed via, generates) are first-class edges rather than implied foreign keys buried in application code.

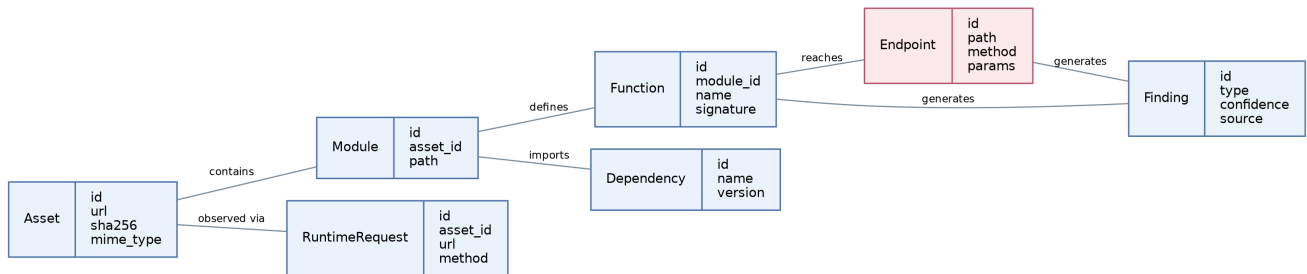


Figure 6. Knowledge graph entities and relationships (entity-relationship view).

The core traversal the platform is built around is: Asset contains Module → Module calls Function → Function reaches Endpoint. Because this is a graph rather than a report, questions like “which assets ultimately reach this endpoint” or “which endpoints would be affected if this dependency were removed” are graph traversals, not custom scripts against export files.

6. Historical Diff Engine

Every scan is stored as a snapshot rather than overwriting the previous one, which lets the Diff Engine (Section 2) answer questions about change over time rather than only current state:

- **First appearance tracking** — the scan run in which an asset, endpoint, or secret was first observed.
- **Endpoint history** — when an endpoint appeared, when its parameters changed, and whether it has since disappeared from the shipped bundle.
- **Dependency changes** — version bumps, additions, and removals of third-party packages between scans.
- **Asset evolution** — hash and size changes to a given asset URL across the scan history.
- **Security posture changes** — net movement in the Risk Scoring Engine's output between two scans, surfaced as a trend rather than a single point-in-time score.

7. Confidence Model

Every finding in JSIntel carries a confidence score derived from the strength of evidence behind it, not a fixed per-analyzer constant. Evidence escalates through four tiers, and a finding confirmed by a later tier inherits the higher confidence:

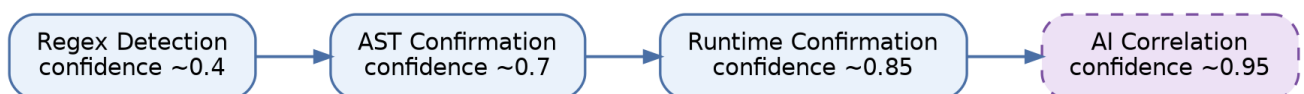


Figure 7. Confidence escalation: regex → AST → runtime → AI correlation.

- **Regex detection** — a pattern match in raw source; cheapest to produce, weakest signal on its own.
- **AST confirmation** — the same signal located structurally (e.g. confirmed as an actual call expression rather than a comment or string).
- **Runtime confirmation** — the behavior was actually observed executing in a browser via the Runtime Intelligence layer.
- **AI correlation** — the JS-specific LLM analyzer corroborates the finding against the full deobfuscated tree, including cross-file context static analysis alone would miss.

8. Automatic API Documentation Generation

Because JSIntel already resolves endpoints, methods, and call sites as first-class knowledge-graph entities, it is positioned to generate API documentation directly from recovered intelligence rather than requiring an operator to write it by hand from scan output.

8.1 Endpoint Reconstruction

For every resolved endpoint, JSIntel recovers as much of the following as the client-side code exposes:

- Path, including path parameters inferred from template literals or route builders.
- HTTP method, from the call site (fetch options, axios method, XHR.open argument).
- Query and body parameters, from the request payload construction.
- Headers, including custom headers set on the request.
- Authentication hints (bearer token headers, cookie-based auth, API key headers).
- Request body structure, where a literal object or typed interface is available.

8.2 OpenAPI Generation

Reconstructed endpoints are assembled into a single `openapi.yaml` document: paths become OpenAPI path items, recovered parameters and headers become parameter objects, and recovered request/response shapes become component schemas where the evidence supports it. Endpoints recovered with low confidence are still included but flagged, so the generated spec is a starting point for review rather than an unqualified source of truth.

8.3 Insomnia REST API Generation

For workflows that want an importable, runnable collection rather than a spec document, the same recovered-endpoint data feeds an Insomnia export pipeline (Figure 6).

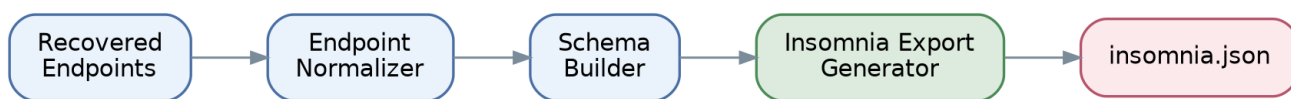


Figure 8. Insomnia collection generation pipeline.

The generator produces:

- Workspaces scoped to a target application.
- Folders grouping requests by resolved feature area (e.g. Authentication, Users, Administration).
- Individual requests with recovered method, path, headers, and parameters pre-filled.
- Environments and variables for base URLs and tokens, rather than hardcoded values per request.
- Authentication templates matching the recovered auth mechanism (bearer, cookie, API key).

Example grouping produced from a single scan:

```
Workspace ■■ Authentication ■ ■■ Login ■ ■■ Refresh Token ■■ Users ■ ■■ Get User
■ ■■ Update User ■■ Administration ■■ Dashboard ■■ Settings
```

At scale — scopes that resolve thousands to millions of endpoints across many assets — the pipeline performs duplicate elimination (same method+path recovered from multiple bundles collapses to one request), automatic grouping by path prefix and resolved feature area, version tracking (an endpoint that changed shape between

scans is versioned rather than overwritten), and environment generation per discovered base URL.

8.4 REST and GraphQL API Layer

Rather than having the HTML dashboard or external tools query SQLite/PostgreSQL directly, a REST/GraphQL API sits in front of a repository layer that owns all data access (Figure 7). This keeps the storage backend swappable and gives external systems a stable integration surface.

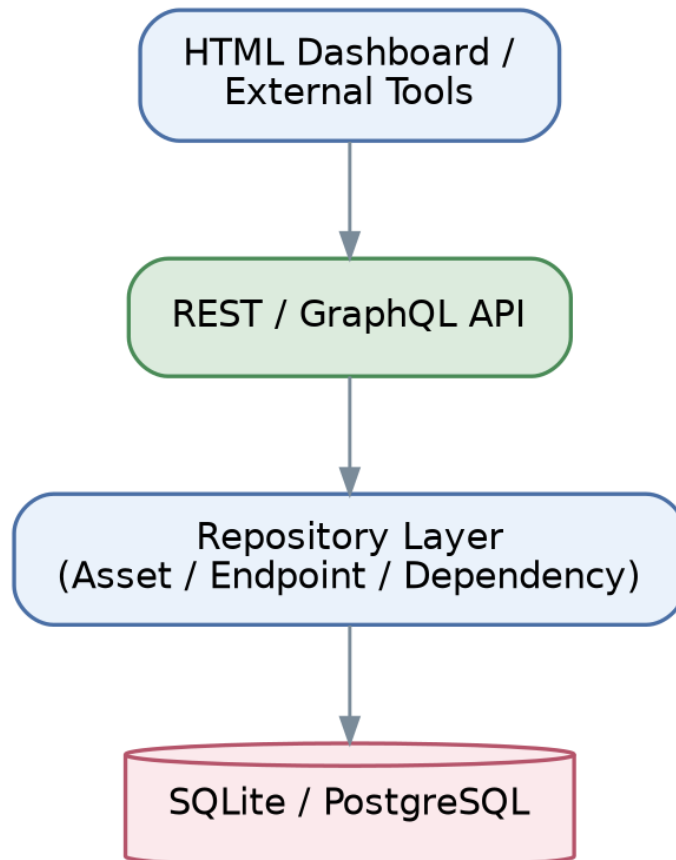


Figure 9. Dashboard and external tools access storage only through the API and repository layer.

This layering is what enables:

- Third-party integrations without direct database coupling.
- Automation scripts driving scans and pulling results programmatically.
- CI/CD pipelines consuming findings as part of a build (Section 9).
- External tools (ticketing, SIEM, asset inventory systems) subscribing to JSIntel data.

9. CI/CD Integration

Findings are exportable as SARIF, the standard format most CI providers already render inline on pull and merge requests. This lets JSIntel participate in the same review workflow as existing SAST tooling instead of requiring a separate dashboard check.

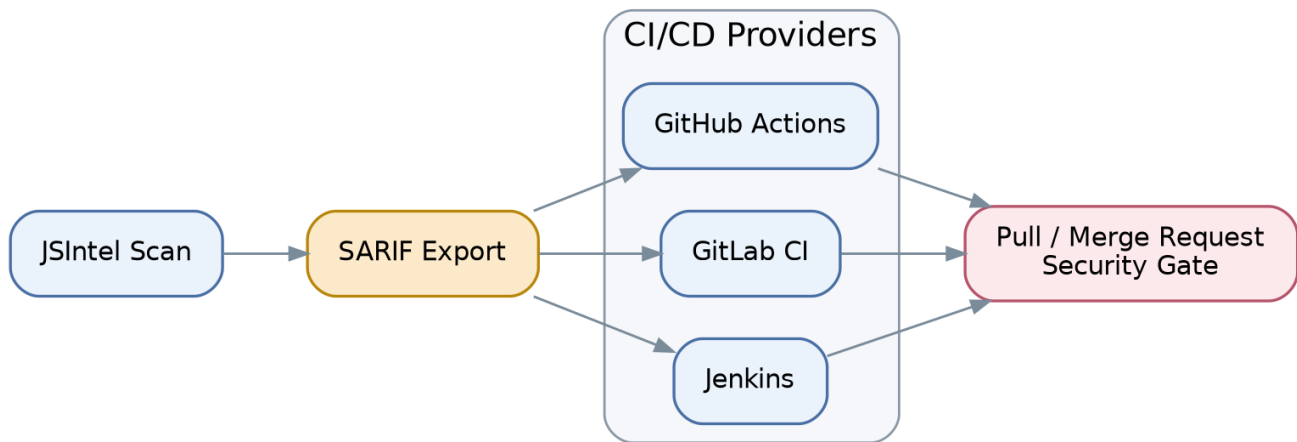


Figure 10. SARIF export feeding pull/merge request security gates across CI providers.

- **GitHub Actions** — SARIF upload via the standard code-scanning upload action, findings annotated directly on the diff.
- **GitLab CI** — SARIF consumed as a GitLab security report artifact for merge request widgets.
- **Jenkins** — SARIF consumed by supporting plugins or archived as a build artifact for downstream gating.

In a pull-request security workflow, a scan runs against the built artifact (or a preview deployment), new or elevated-severity findings relative to the target branch's last scan are surfaced as review comments, and the build can be configured to fail on findings above a confidence and severity threshold rather than on any finding at all — avoiding noise from low-confidence regex-only hits.

10. Enterprise Roadmap

- **Multi-tenancy** — scoped scan data per team or engagement within one deployment, so results from concurrent authorized engagements do not mix.
- **Distributed workers** — horizontally scaled crawl and analysis workers coordinating through a shared queue, for scopes too large for single-host scanning.
- **PostgreSQL backend** — the existing optional PostgreSQL target (Section 2) as the default backing store for multi-user and multi-tenant deployments.
- **Cloud deployment** — containerized deployment (the existing Docker integration) onto managed compute for teams that do not want to operate infrastructure.
- **Continuous monitoring** — scheduled re-scans of a fixed scope with diff-aware AI analysis (Section 4.1) so drift is detected between engagements, not only during them.

11. Plugin Contract

All intelligence modules in the Analysis Engine are implemented as plugins against a fixed four-method contract, which is what lets AST analyzers, the taint engine, and the AI layer coexist without the Plugin Manager needing special cases for any of them:

- **initialize()** — prepare plugin state and load configuration.
- **analyze()** — process a given asset (or, for the AI layer, a change set) and emit typed findings.
- **finalize()** — flush buffered results and release resources.
- **schema()** — declare the structure of findings the plugin produces, validated against the same finding shape shown in Section 4.2.

12. Database Architecture

The knowledge layer's default backing store is SQLite, with PostgreSQL supported for larger or multi-user deployments (Section 10). The core tables underlying the knowledge graph in Section 5 are summarized below.

Table	Key Columns	Purpose
assets	id, url, sha256, mime_type, size, fetched_at	One row per downloaded artifact and its integrity metadata.
modules	id, asset_id, path	Resolved module boundaries within a parsed asset.
functions	id, module_id, name, signature	Function/class declarations extracted from the AST.
endpoints	id, path, method, params, confidence	Reconstructed API endpoints and recovered request shape.
dependencies	id, name, version	Third-party packages identified via import/dependency analysis.
runtime_requests	id, asset_id, url, method, observed_at	Network calls captured via browser instrumentation.
findings	id, type, confidence, source, value	Typed output from any analyzer plugin, including the AI layer.
scan_history	id, scope, started_at, completed_at	Snapshot boundary used by the Historical Diff Engine.

13. Installation & Usage

On Kali Linux or Debian:

```
chmod +x install.sh jsintel.sh modules/*.sh modules/*.py
./install.sh
```

The installer places Katana and HTTPX under the Go bin path. Playwright's browser runtime is only required for runtime intelligence (Section 2) — it is not needed for the Phase 1 Katana crawler.

13.1 Running a Scan

Create a scope file with one authorized seed URL per line, then run the pipeline:

```
./jsintel.sh -i crawled_urls -o target_output -t 50
```

The `-o` flag defaults to `./output`; `-t` reads thread/timeout defaults from `config/config.yaml` (50). A single URL may also be passed directly to `-i`.

13.2 Output Layout

```
target_output/
  assets/           # crawled_urls.txt, downloaded assets
  database/recon.db
  logs/jsintel.log
  reports/          # assets.json, urls.json, endpoints.json,
                    # websocket.json, imports.json, frameworks.json,
                    # assets.csv, summary.json, summary.md
```

Ad-hoc queries are available through the database module, for example: `python3 modules/database.py --output target_output query 'SELECT endpoint FROM endpoints'`.

13.3 Project Layout

```
jsintel-phase1/
  jsintel.sh        # Pipeline launcher
  install.sh        # Debian/Kali dependency installation
  modules/          # Crawl, classify, download, extract, database, report
  database/schema.sql # SQLite schema
  config/config.yaml # Runtime defaults
  reports/          # Checked-in report exports
  output/           # Default runtime output root
```

14. Operational Notes

- The crawler uses Katana when it is installed; if unavailable, it records the supplied HTTP(S) URLs as candidate assets rather than crawling pages — useful for offline or constrained environments.
- Downloads are limited to discovered HTTP(S) URLs and are saved using sanitized filenames.
- One malformed or missing asset no longer aborts a full scan; errors are captured in `errors.json`.
- JSIntel should never be run outside a scope the operator is explicitly authorized to assess.